



PDF Download
3731569.3764849.pdf
29 March 2026
Total Citations: 0
Total Downloads: 3863



Published: 12 October 2025

Citation in BibTeX format

SOSP '25: ACM SIGOPS 31st Symposium
on Operating Systems Principles
October 13 - 16, 2025
Seoul, Republic of Korea

Conference Sponsors:
SIGOPS

DL Latest updates: <https://dl.acm.org/doi/10.1145/3731569.3764849>

RESEARCH-ARTICLE

DCP: Addressing Input Dynamism In Long-Context Training via Dynamic Context Parallelism

CHENYU JIANG, The University of Hong Kong, Hong Kong, Hong Kong

ZHENKUN CAI, Amazon.com, Inc., Seattle, WA, United States

YE TIAN, The University of Hong Kong, Hong Kong, Hong Kong

ZHEN JIA, Amazon.com, Inc., Seattle, WA, United States

YIDA WANG, Amazon.com, Inc., Seattle, WA, United States

CHUAN WU, The University of Hong Kong, Hong Kong, Hong Kong

Open Access Support provided by:

The University of Hong Kong

Amazon.com, Inc.



DCP: Addressing Input Dynamism In Long-Context Training via Dynamic Context Parallelism

Chenyu Jiang*

jchenyu@connect.hku.hk
The University of Hong Kong

Zhenkun Cai

zkcai@amazon.com
Amazon Web Services

Ye Tian*

yetiansh@connect.hku.hk
The University of Hong Kong

Zhen Jia

zhej@amazon.com
Amazon Web Services

Yida Wang

wangyida@amazon.com
Amazon Web Services

Chuan Wu

cwu@cs.hku.hk
The University of Hong Kong

ABSTRACT

Context parallelism has emerged as a key technique to support long-context training, a growing trend in generative AI for modern large models. However, existing context parallel methods rely on static parallelization configurations that overlook the dynamic nature of training data, specifically, the variability in sequence lengths and token relationships (i.e., attention patterns) across samples. As a result, these methods often suffer from unnecessary communication overhead and imbalanced computation. In this paper, we present DCP, a dynamic context parallel training framework that introduces fine-grained blockwise partitioning of both data and computation. By enabling flexible mapping of data and computation blocks to devices, DCP can adapt to varying sequence characteristics, effectively reducing communication and improving memory and computation balance. Micro-benchmarks demonstrate that DCP accelerates attention by 1.19x~2.45x under causal masks and 2.15x~3.77x under sparse attention patterns. Additionally, we observe up to 0.94x~1.16x end-to-end training speed-up for causal masks, and 1.00x~1.46x for sparse masks.

CCS CONCEPTS

• **Computing methodologies** → **Distributed computing methodologies**; *Artificial intelligence*.

*Work done during internship at AWS.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SOSP '25, October 13–16, 2025, Seoul, Republic of Korea
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1870-0/25/10
<https://doi.org/10.1145/3731569.3764849>

KEYWORDS

large language models, attention mechanism, distributed model training, context parallelism

ACM Reference Format:

Chenyu Jiang, Zhenkun Cai, Ye Tian, Zhen Jia, Yida Wang, and Chuan Wu. 2025. DCP: Addressing Input Dynamism In Long-Context Training via Dynamic Context Parallelism. In *ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25)*, October 13–16, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3731569.3764849>

1 INTRODUCTION

State-of-the-art AI systems have achieved remarkable performance across a diverse range of tasks [4, 11, 19, 22, 32]. A notable trend in modern large models is the increasing context length (number of tokens as input), meant for enhancing deep learning models' capacity in processing extensive amounts of information (e.g., long documents and code-bases). For example, GPT-4o supports a 128K context window [32]; Claude 3.5 Sonnet extends the context window size to 200K [4]; Gemini 2.5 Pro scales the context window to 2M tokens [19]. The increased context length greatly raises the memory and computation requirements of state-of-the-art large generative models, making them significantly more expensive to train.

To address this challenge, recent approaches adopt context parallelism (CP), which partitions each sequence in the training data evenly across all devices [14, 20, 23, 28]. These methods reduce memory consumption and enable training with longer context lengths, but incur additional communication overhead [13, 47]. Notably, this communication cost increases with the size of the training cluster (Fig. 1). As both model sizes and context lengths continue to grow, the communication overhead is expected to increase significantly.

Existing CP approaches uniformly apply a fixed parallelization configuration (data partitioning and placement) for all batches. Such static partitioning methods fail to account for the inherent dynamism in training data, which we categorize into: 1. the variance in input sequence lengths,

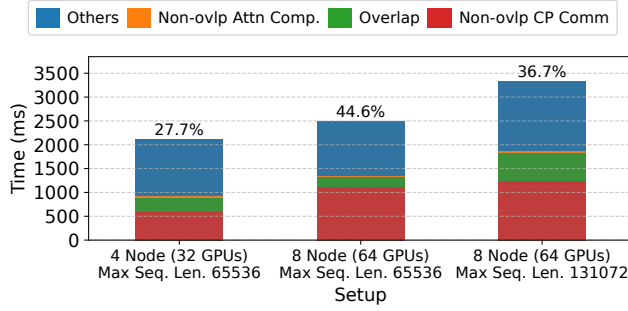


Figure 1: CP communication overhead when training a 8B GPT model on an Amazon EC2 p4d.24xlarge cluster (400Gbps interconnect between nodes) with 4-way tensor and 16-way context parallelism, using the LongAlign [5] dataset. Overlap: overlapping CP communication and attention computation. Communication overhead fraction (vs. total iteration time) is shown above each bar.

and 2. the variance in token relationships within each sequence. As a result, these methods miss key opportunities for optimization.

Variance in input sequence lengths. Modern training datasets often exhibit a highly skewed distribution of sequence lengths, especially in long-context settings, where shorter sequences are significantly more common than longer ones [18, 24]. For instance, during the supervised fine-tuning phase of Llama 3 training, long-context samples constitute only 0.11% of the dataset [13]. Similar patterns can be observed in other datasets (see Fig. 2). Larger pre-training datasets, such as The Pile [16], also exhibit similar document length distributions. Static parallelization can introduce redundant communication when processing shorter sequences, thereby increasing overall execution time.

Variance in token relationships (attention patterns). In modern LLMs, token relationships are typically expressed through attention masks. Existing static context parallelization schemes are primarily designed for causal attention [14, 20, 28]. However, recent studies have advocated the use of diverse attention masks to accelerate training or address novel training scenarios. For example, in reinforcement learning-based post-training, a shared question mask can eliminate redundant computation between a question and its multiple answers [43]. Sliding-window or lambda-shaped masks [21, 27] are widely used to significantly sparsify attention and reduce required computation. These sparse or structured attention masks break the assumptions on attention workload distribution that static methods rely on. As a result, applying static partitioning in such settings leads to severe load imbalance and redundant communication, which undermines performance.

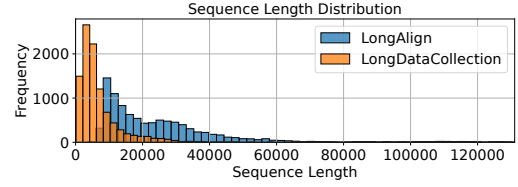


Figure 2: Sequence length distribution of LongAlign [5] and LongDataCollection [41] datasets, capped at 131072.

To address the limitations of static parallelization configurations in current context parallelism approaches, we propose a dynamic context parallelism approach that constructs a different parallelization configuration for each training iteration. We systematically model context-parallel training by partitioning attention inputs and outputs into fine-grained data blocks and constructing computation blocks that capture attention patterns. These blocks can be flexibly assigned to different devices, enabling customized parallelism and data/computation placement configurations tailored to each sequence. We optimize the placement of data and computation blocks in each iteration by formulating it as a hypergraph partitioning problem, aiming to minimize communication costs while satisfying memory and compute balance constraints. We also automatically generate computation and communication schedules for the blocks assigned to each device, forming pipelines to overlap their execution. This dynamic planning is managed by a data loader wrapper, which pre-fetches data and serializes device-specific schedules using five distinct instructions prior to the corresponding iteration. A custom executor efficiently executes these instructions using fused kernels, minimizing the overhead associated with fine-grained parallelism.

We summarize our contributions as follows:

- We devise a representation for both data and computation that explicitly captures the effect of input dynamism in context parallelism, allowing us to systematically define fine-grained parallelization configurations for each training iteration.
- We formulate the problem of optimizing the parallelization configuration with hypergraph partitioning, enabling efficient solutions using established algorithms [38].
- We provide an end-to-end framework implementation (i.e., DCP) that enables long context training with dynamic context parallelism, minimizing planning and runtime overheads.
- We perform an extensive evaluation against state-of-the-art CP frameworks including TransformerEngine [30] and LoongTrain [20]. Micro-benchmarks show that DCP achieves 1.19x~2.45x speed-up with causal, and 2.15x~3.77x with sparse attention masks for individual attention layers. End-to-end experiments show a 0.94x~1.16x speed-up with causal mask, and 1.00x~1.46x with sparse attention masks.

2 BACKGROUND AND MOTIVATION

2.1 Block-wise attention computation

Attention is one of the central components in transformer-based large models [42]. The standard masked self-attention is:

$$O_{bh::} = \text{RowWiseSoftmax}\left(\frac{(Q_{bh::}K_{bh::}^T) \odot M_{bh::}}{\sqrt{D}}\right)V_{bh::}$$

where Q (query), K (key), V (value), O (attention output) are 4-dimensional tensors of shape $[B, H, L, D]$. B is the input batch size, H is the number of attention heads, L is the input sequence length and D is head dimension size. Subscripts $bh::$ are indices into the respective dimensions, i.e., $Q_{bh::}$ is a slice (matrix of shape $[L, D]$) of the tensor Q at index b in the first dimension (B) and h in the second dimension (H). M is a boolean mask of shape $[B, L, L]$, zeroing out unwanted interactions between tokens during attention computation (i.e., the attention mask). Using the online softmax trick [10, 34], attention can be computed block-wise, with each block processed in parallel. Suppose that we divide each tensor into $\mathcal{B}_b$ blocks along the batch dimension and $\mathcal{B}_h$ blocks along the head dimension; then, along the sequence length dimension, we divide Q into $\mathcal{B}_q$ blocks, and K, V into $\mathcal{B}_{kv}$ blocks, the parallel attention computation can be expressed in pseudo code as:

Listing 1: Block parallel attention

```

1 parallel for b in [1, Bb]:
2   parallel for h in [1, Bh]:
3     parallel for q in [1, Bq]:
4       parallel for k in [1, Bkv]:
5          $\hat{O}_{bhqk} = \text{Softmax}\left(\frac{(Q_{bhq}K_{bhk}^T) \odot M_{bhqk}}{\sqrt{D}}\right)V_{bhk}$ 
6          $O_{bhq} = \text{RescaleAndSum}(\hat{O}_{bhq1}, \dots, \hat{O}_{bhqB_{kv}})$ 

```

where the subscripts are now block indices. Such block-wise parallelization is widely adopted by efficient GPU attention kernel libraries like FlashAttention [10], as it eliminates the memory access cost of materializing the large intermediate tensor QK^T . With appropriate block sizes, Q or KV blocks can also reside in the fast GPU shared memory, enabling efficient reuse and further reducing memory access costs. Therefore, we also use such block division design to analyze distributed attention and identify the following four parallelizable dimensions for the attention operator: **batch**, **head**, **SeqQ** and **SeqKV**, corresponding to the first four lines in Listing 1, respectively (Fig. 3). $\hat{O}_{bhqk}$ (Line 5) with different subscripts can potentially be computed in parallel across devices. Since K and V are always partitioned and parallelized together, we refer to them as a whole as KV , in the rest of the paper.

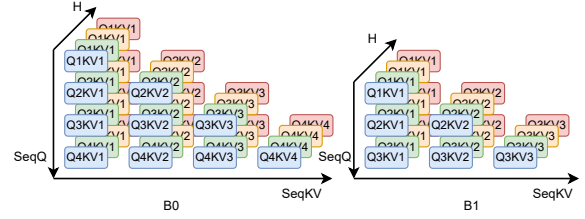


Figure 3: Four parallelizable dimensions of an attention operator. Each block represents the computation of $\hat{O}_{bhqk}$ (Listing 1 line 5) with corresponding Q and KV blocks. Figure shows attention of two sequences (B0 and B1) with lengths 4 and 3 tokens, respectively, using causal mask.

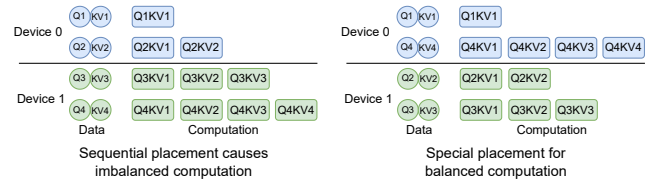


Figure 4: Special data and computation placement for causal mask.

2.2 Context parallelism

Operators in a transformer-based large model can be categorized into: 1) attention operators, whose computation of tokens depends on other tokens in an input sequence; 2) context-independent operators, in which computations of tokens are independent (e.g., layer normalization and MLP layers). Context parallelism of an operator parallelizes the computation of tokens within each input sequence among devices. Context parallelizing the attention operator must be handled carefully, as memory/computation balance and communication overhead are all critical to performance. Context parallelizing context-independent operators is more straightforward, as it does not involve additional communication overhead during parallel token processing by different devices. Context parallelism's token-to-device assignment is usually inherently decided by input and output data (token) placement of connected attention layers.

Existing context parallelism performs distributed attention at the head and SeqQ dimensions, with fixed communication schedules as well as input data and computation placements. When parallelizing at the SeqQ dimension, each device is responsible for the computation of certain tokens in a query with all KVs, with the communication for KV forming a ring pattern [28, 49]. When parallelizing at the head dimension, each device calculates different attention heads and needs to access Q , KV and output of assigned heads of all sequences,

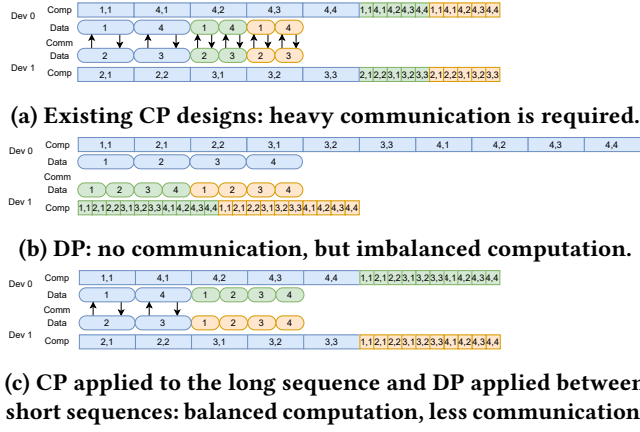


Figure 5: Communication and computation required under different parallelization configurations. Color distinguishes different sequences. Round-cornered blocks: input token blocks (4 blocks in each sequence), where each block in the blue sequence contains twice as many tokens as green and orange sequences. Square blocks $[a, b]$: attention between token block a and token block b . Arrow: communication of KV for corresponding blocks.

resulting in all-to-all communications [23]. The two parallelization schemes can also be applied jointly [14, 20, 30]. Most methods [14, 20, 30, 49] use a placement that optimizes computation balance under the causal mask: With R devices, each input sequence is uniformly sliced into $2R$ chunks, and the i th device is assigned the i th chunk and the $(2R - i + 1)$ th chunk (Fig. 4).

2.3 Redundant communication on variable-length inputs

We observe communication inefficiency with current context parallelism (CP) designs, under variable input sequence lengths and number of input sequences per training batch. Under the existing CP designs, CP degree is fixed throughout the entire model training, i.e., each input sequence is partitioned into an equal number of parts, which are then evenly distributed among devices. While such partitioning guarantees memory and computation balance, communication for exchanging KV is required for every input sequence, regardless of the sequence length (Fig. 5a).

One way to improve the communication efficiency is to allow dynamic adjustment of CP and data parallelism (DP) degrees. However, it is hard to simultaneously balance computation and memory across DP groups, since the memory requirement grows linearly with the number of assigned tokens [10, 25], while computation grows quadratically [10]. In Fig. 5b, the long sequence is placed on Device 0 and the

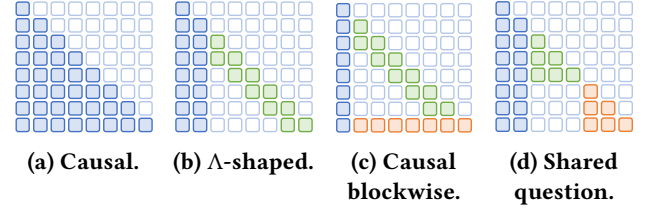


Figure 6: Attention masks used in various applications. Colors highlight different attention mask components.

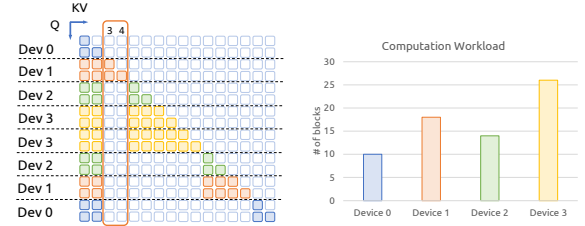


Figure 7: Imbalanced computation and redundant communication when applying ring attention on a shared question mask [43].

(b) Communication of redundant KV blocks (those not used by the receiving device) are underlined and in bold font. Numbers on the communication arrows indicate KV block indices.

Figure 7: Imbalanced computation and redundant communication when applying ring attention on a shared question mask [43].

shorter ones on Device 1, achieving perfect memory balance and eliminating all CP communication, but resulting in heavy computation imbalance between devices.

Better performance can be achieved if we allow more fine-grained parallelization configurations (e.g., applying different types of parallelism for different input sequences). In Fig. 5c, CP is applied to the long sequence and DP to the rest; both memory and computation are perfectly balanced, while communication is reduced by half compared to using pure CP, leading to improved execution time in communication-bound scenarios.

2.4 Inability to adapt to diverse attention masks

While current methods design special placement only for the causal mask (Fig. 4), more sparse attention masks are used in long-context model training and inference (Fig. 6). Lambda(Λ)-shaped mask (Fig. 6b) combines attention sink

(all tokens attend to several tokens at the start of the sequence) and sliding window attention (each token only attends to a predefined number of precedent tokens), and is widely applied to reduce the total attention computation and improve training/inference speed [21, 46]. Causal blockwise mask (Fig. 6c) reduces attention computation while maintaining model performance in in-context learning (ICL) scenarios [6]: the input is divided into blocks (each containing multiple ICL examples), attention sink and sliding window attention are applied to these blocks, and the final test example attends to all previous tokens. In RLHF [33] or DPO [36] training where each question may be paired with multiple candidate answers, a shared question mask (Fig. 6d) can be used to allow answers to share the same prefix question and thus remove redundant computation [43]. Notably, in causal blockwise mask and shared question mask, the shape of the attention mask is determined not only by the model design, but also by the input data, and thus different attention masks are applied to different input batches. These sparse masks are not supported by current context parallelism frameworks [14, 20, 30, 49].

To perform distributed attention computation under these masking patterns, a simple approach is to retain current placement and communication patterns while applying attention masks to each of the local attention steps. However, doing so may cause significant computation imbalance and communication redundancy. In the example with a shared question mask in Fig. 7, current methods assign far more computation workload to device 3 (yellow blocks) than to the other devices. Due to the mask’s sparsity, the KV for token block 3-4 (columns highlighted in Fig. 7a) is only required on Device 1, while the current communication design requires it to be transferred to all devices. As a result, 38 out of 48 KV blocks (16 per step, over 3 steps) are redundantly communicated (see Fig. 7b).

2.5 Opportunities and Challenges

To mitigate such issues caused by dynamic and flexible model input, we need fine-grained parallelism control across batches as well as for each input sequence within a batch. Such fine-grained parallelism configuration should minimize communication while maintaining memory and computation balance. This calls for solving the following challenges:

- How to capture the effect of dynamism on context parallelism and systematically define the fine-grained parallelization configuration?
- How to automatically generate optimized parallelization configurations for different batches?
- How to efficiently implement such dynamic and fine-grained parallelism, with potentially different parallelization configurations for each iteration?

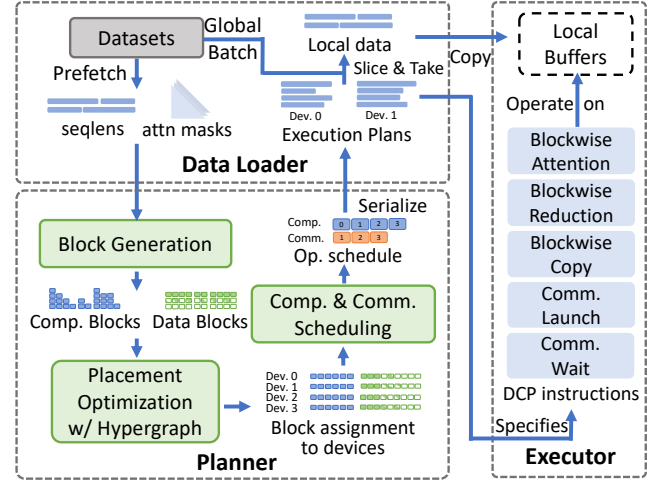


Figure 8: DCP System Overview

3 OVERVIEW

We design DCP, a context parallelism framework for efficient training of large models that dynamically adapts parallelization configurations to arbitrary sequence lengths and varying attention masks.

3.1 Workflow

The workflow of DCP is given in Fig. 8. It consists of three key modules.

The data-loader pre-fetches sequence lengths and mask patterns from training datasets. For each input batch, it performs data partitioning, generates data and computation blocks that describe context parallelism for the input batch (§4.1) based on sequence length and mask information, and invokes the planner. Based on the data placement generated by the planner, it constructs the local model inputs for each device.

The planner solves a hypergraph partitioning problem to optimally assign data and computation to devices. It then performs computation and communication scheduling and generates the execution plan for each device. The execution plans are serialized in the form of DCP instructions (Sec. 5), each abstracting an elementary operation required to execute attention. The above planning is carried out online during training and overlaps with GPU training through data pre-fetching from the datasets.

The executor implements the DCP instructions with high-performance kernel libraries (e.g., FlashAttention [10]) or compilers like Triton [40]. During each iteration, it creates a buffer storing inputs/outputs fetched from other devices and intermediate results on each device. Then, it sequentially carries out the instructions specified by the execution plan.

3.2 User Interface

DCP provides a simple user interface for easy integration by developers training large models, as given in Listing 2. To use DCP, developers begin by replacing the attention implementation in their models with DCP’s implementation (Lines 1-7). If non-causal attention masks are used, users can define a custom mask-generation function (Lines 9-12) that takes input information (available from the dataset) like sequence lengths and their composition (e.g., lengths of questions and answers when using the shared question mask). In their training script, users construct a data-loader (Line 15) with the dataset and the mask function, and an executor (Line 18) which is shared across all model layers. In each training iteration, users can directly get the partitioned data and corresponding execution plan from the data-loader for each device (Line 20), initialize the executor with the execution plan (Line 22), and run the model training iteration (Line 24).

Listing 2: DCP API

```

1 # when defining models
2 class TransformerLayer(...):
3     def forward(..., dcp_executor):
4         ...
5         # replace attention implementation with DCPAttn
6         core_attn_out = DCPAttn.apply(dcp_executor, q, kv)
7         ...
8
9 # define a mask function
10 def mask_fn(seqlens, ...)
11     ...
12     return mask
13
14 # in training script
15 dcp_data_loader = DCPDataLoader(dataset, mask_fn)
16 # dcp_group is a communicator that connects all devices
17 # (e.g., torch.distributed.ProcessGroup)
18 dcp_executor = DCPExecutor(group=dcp_group)
19 # training iterations
20 for (local_data, execution_plan) in dcp_data_loader:
21     # set execution plan and create buffers
22     dcp_executor.prepare(execution_plan)
23     # execute model
24     loss = model(local_data, dcp_executor)
25     ...

```

4 DYNAMIC CONTEXT PARALLELISM

DCP advocates dynamically optimizing parallelism configurations for each training batch to best handle varying sequence lengths and attention masks, and enables switching between different configurations across training batches/iterations (i.e., *dynamic context parallelism*). To identify optimized configurations, we define a unified representation for the parallelization of a training batch. In this approach, we divide both the data and computation of each input sequence

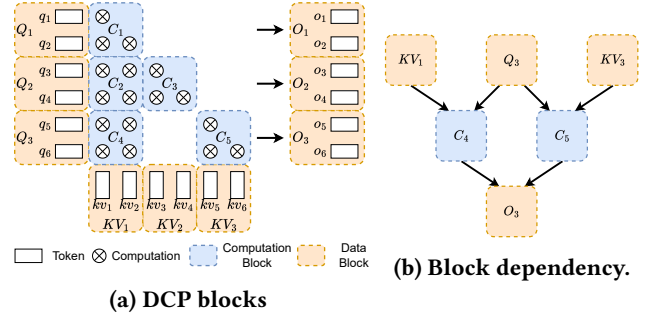


Figure 9: Data and computation block partitioning.

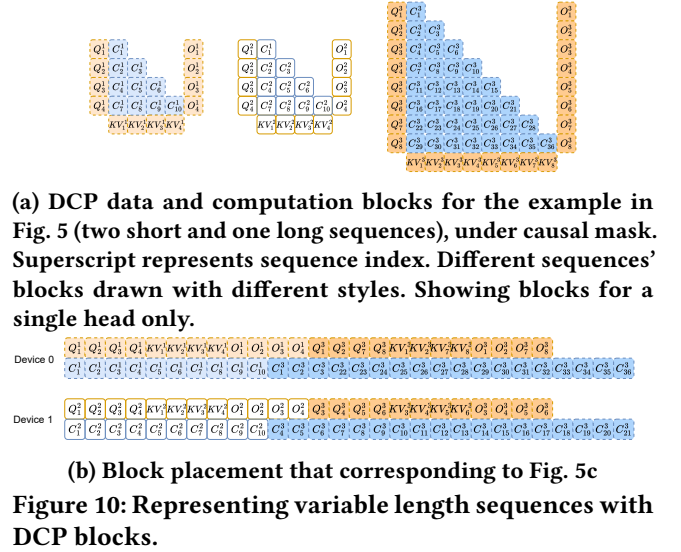


Figure 10: Representing variable length sequences with DCP blocks.

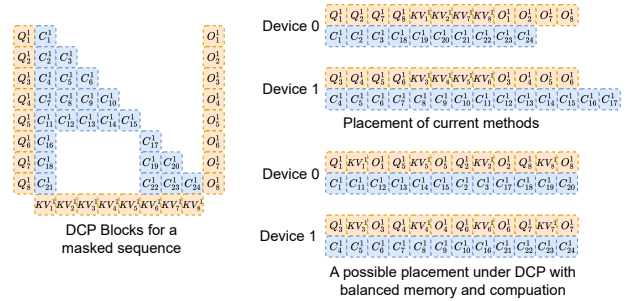


Figure 11: Example of DCP blocks for a shared question masked sequence, with two different possible placements. Showing blocks for a single head only.

into blocks along all parallelizable dimensions. A parallelism configuration is then defined by assigning these blocks to devices, which in turn determines the communication pattern between devices.

4.1 Block Generation

For each sequence in an input batch, we partition the input and output tensors of attention operators at every parallelizable dimension into contiguous slices, and denote each

resulting slice as a *data block*. The input tensors are $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ and the output tensor is $\mathbf{O}$, each of shape $[H, L, D]$ (where H is the number of heads, L is the sequence length, and D is the hidden dimension). Each tensor is partitioned at head and sequence length dimensions, where each partitioned block is of shape $[1, \mathcal{B}, D]$, with $H \times \frac{L}{\mathcal{B}}$ blocks in total. Here block size $\mathcal{B}$ is a hyper-parameter of the algorithm.

Computation executed by the attention operator is decomposed into *computation blocks*, each describing the attention between a query block $\mathbf{Q}_i$ and a pair of key and value blocks $\mathbf{KV}_j$, which contributes to an output block $\mathbf{O}_i$, representing the computation of $\hat{O}_{bhij}$ in Line 5 of Listing 1 (here b and h are sequence and head index for blocks $\mathbf{Q}_i$ and $\mathbf{KV}_j$). Each of the computation blocks represents computation that can be executed in parallel. Fig. 9a shows the data and computation blocks for a single sequence of 6 tokens under a shared question mask [43] (showing blocks for a single head only). Each of $\mathbf{Q}$, $\mathbf{KV}$, and $\mathbf{O}$ is partitioned into three blocks (i.e., $\mathcal{B} = 2$), respectively. For each pair of $\mathbf{Q}_i$ and $\mathbf{KV}_j$ with computation dependency (i.e., the corresponding attention mask $\mathbf{M}_{bij}$ is not all zeros), a computation block $\mathbf{C}_k$ is constructed and contributes to output block $\mathbf{O}_i$. Fig. 9b shows the data dependencies of computation blocks $\mathbf{C}_4$ and $\mathbf{C}_5$ in Fig. 9a. When multiple computation blocks contribute to the same output block, a reduction is required to aggregate the results (Lst. 1, Line 6).

We allow arbitrary device assignment of data and computation blocks, with the constraint that the blocks corresponding to $\mathbf{Q}, \mathbf{KV}$ and $\mathbf{O}$ of the same tokens are placed onto the same device (since the input batch is partitioned across devices at the token level). The placement of these $\mathbf{Q}, \mathbf{KV}$ and $\mathbf{O}$ data blocks thus determines the device assignment of the corresponding tokens (i.e., we use these tokens as the input to the model replica on that device), while the placement of a computation block determines the device where the corresponding attention computation is performed. When a computation block and the required input blocks ($\mathbf{Q}, \mathbf{KV}$) are assigned to different devices, communication occurs to fetch the data blocks to the computation device (and vice versa for output blocks).

With such fine-grained representation, we can easily control the parallelization configuration for each input sequence, regardless of its length. Fig. 10 shows the data and computation blocks corresponding to the three sequence examples in Fig. 5. To represent the parallelism configuration of current approaches, we assign half of data and computation blocks of each sequence to each device. For a pure DP configuration, we assign all blocks of sequence 1, 2 to device 0, and all blocks of sequence 3 to device 1. The configuration of Fig. 5c (using DP for the two short sequences while using CP for the long sequence) can be achieved by assigning all blocks

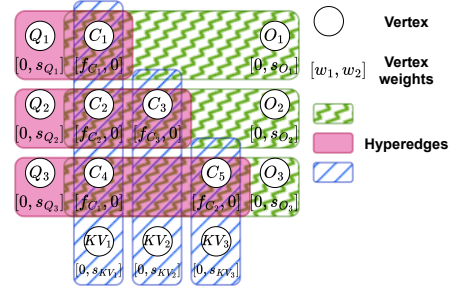


Figure 12: Hypergraph for the example in Fig. 10.

of sequence 1 to device 0, all blocks of sequence 2 to device 1, and half of data and computation blocks of sequence 3 to each of the devices (Fig. 10b).

This fine-grained representation of attention computation also effectively discards unnecessary computation (i.e., where attention mask $\mathbf{M}_{bij} = 0$), as those blocks will not be constructed. The flexible block assignment makes it easy to load-balance memory and computation for masked attention. Fig. 11 shows the blocks generated for a sequence with shared question mask [43] and possible device placements. The blank area between computation blocks $\mathbf{C}_{16}^1$ and $\mathbf{C}_{17}^1$, $\mathbf{C}_{18}^1$ and $\mathbf{C}_{19}^1$, $\mathbf{C}_{21}^1$ and $\mathbf{C}_{22}^1$ are computations that are masked out. Load balancing in DCP is achieved by assigning data/computation blocks that represent similar total data size/computation FLOPS to each device.

Such representation further facilitates modeling of communication volume incurred in any given context parallelization configuration. Consider attention computation on an input sequence through a set of computation blocks $\mathbf{C}$. Each computation block $\mathbf{C}_i$ consumes input data blocks $\mathbf{I}_i$ (including $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ blocks) and contributes to output data blocks $\mathbf{O}_i$, and each data or computation block is assigned to a device $\text{Dev}(\cdot)$. Communication is incurred for fetching remote data blocks to the device where the computation is located, and sending output blocks to the device where the output is needed. The total communication volume under this configuration is $\sum_{i=1}^{|\mathbf{C}|} \left(\sum_{j=1}^{|\mathbf{I}_i|} \text{Size}(\mathbf{I}_{ij}) \cdot [\text{Dev}(\mathbf{I}_{ij}) \neq \text{Dev}(\mathbf{C}_i)] + \sum_{j=1}^{|\mathbf{O}_i|} \text{Size}(\mathbf{O}_{ij}) \cdot [\text{Dev}(\mathbf{O}_{ij}) \neq \text{Dev}(\mathbf{C}_i)] \right)$ where $\text{Size}(\cdot)$ denotes the size of the data block and $[x]$ is the Iverson bracket notation, equaling 1 if the condition x is true, and 0, otherwise.

4.2 Hypergraph partitioning for data and computation placement

Device placement of data and computation blocks decides communication overhead in context parallelism, as well as computation load and memory consumption on devices for both context-independent (e.g., layer normalization, MLP)

and attention operators. Computation load and memory usage of context-independent operators depend on the number of tokens processed on a device, which is in turn determined by our assignment of data blocks. Thus we do not separately model the memory and computation of context-independent operators; by ensuring balanced data block distribution (for attention) across devices, we automatically enable balanced context-independent operator computation and memory consumption.

Given that intra-machine communication usually has much higher bandwidth and lower latency than inter-machine communication, it is beneficial to prioritize minimizing inter-machine communication volume. We design an efficient *hierarchical placement* scheme accordingly. On a cluster of X machines and Y devices per machine, we first assign blocks to the X machines, aiming to minimize cross-machine communication. Next, we further optimize the placement of blocks on each machine onto the Y devices.

For each level of placement, we describe distributed computation of an attention operator on an input batch with a hypergraph $G = (N, E)$. Hypergraphs differ from normal graphs in that each hyperedge $e \in E$ can connect more than two vertices in N , enabling the modeling of complex dependencies in applications like sparse matrix-vector multiplication [9]. To model distributed attention, let $N = C \cup I \cup O$ denote the set of vertices, comprising computation blocks, input and output data blocks for all sequences in an input batch. The set of hyperedges E is defined such that each hyperedge $e \in E$ connects a data block d to the set of computation blocks $\{C_i | d \in I_i \text{ or } d \in O_i\}$ (i.e., all computation blocks that either consume or contribute to it). Each vertex is associated with a 2-dimensional weight, where the first weight value represents computation load and the second indicates data size. The weight of each computation block $c \in C$ (vertex n_c) is $\mathbf{w}_{n_c} = [f_{n_c}, 0]$ where f_{n_c} represents the amount of computation, e.g., FLOPS, of the computation block. The weight of each data block $d \in I_i$ or O_i (vertex n_d) is $\mathbf{w}_{n_d} = [0, s_{n_d}]$ where s_{n_d} is the size of the data block. Each hyperedge e has weight s_e , representing data size of the data block in the hyperedge. Fig. 12 illustrates the formulated hypergraph of Fig. 9a. Nine hyperedges are built in total, connecting each data block to its associated computation blocks.

Given R devices (i.e., $R = X$ for machine-level placement and $R = Y$ for device-level placement), we divide vertices in the hypergraph into R balanced partitions, $P_1, \dots, P_R$, by balancing the total vertex weight among partitions while minimizing a connectivity metric, and assign each partition to a device. The connectivity metric is $\sum_{e \in E} s_e (\lambda_e - 1)$ (λ_e denotes the number of partitions that vertices in hyperedge e span), representing the weighted sum of each hyperedge's connectivity minus one. When vertices connected by a hyperedge e are assigned to λ_e different devices, $s_e (\lambda_e - 1)$

represents the total communication volume of the data block in hyperedge e . Therefore, the connectivity metric models the total communication volume required in the resulting hypergraph partitioning [9].

The hypergraph partitioning and block-to-device assignment problem is:

$$\min_{P_1, \dots, P_R} \sum_{e \in E} s_e (\lambda_e - 1)$$

subject to

$$\mathbf{w}(P_i) \preceq [1 + \epsilon, 1] \odot \frac{\mathbf{w}(N)}{R}, \forall i \in [1, R]$$

where $\mathbf{w}(P_i) = \sum_{n \in P_i} \mathbf{w}_n = [\sum_{n \in P_i} f_{n_c}, \sum_{n \in P_i} s_{n_d}]$ is the total vertex (computation and data) weight in P_i , $\preceq$ denotes less or equal in both weight dimensions, and $\odot$ denotes element-wise multiplication. ϵ is a small positive value specifying the computation imbalance tolerance (we always try to make data blocks as balanced as possible). The constraint ensures computation and memory balance across partitions. This balanced hypergraph partitioning problem is NP-hard [17], with efficient heuristics available (e.g., multi-level partitioning [8]), implemented by off-the-shelf solvers such as PaToH [7] and KaHyPar [38].

4.3 Computation & communication scheduling

Placement of data and computation blocks determines the necessary communication between devices. However, sequentially executing assigned computation blocks on each device, along with their associated communication, may not saturate hardware resources, including both computing power and bandwidth. To enhance hardware utilization and minimize communication overhead, we advocate a multi-division execution schedule, by grouping computation block on each device into divisions and overlapping the computation of one division with the communication of the next, where communication and computation for each division is executed with fused kernels.

Computation blocks from a training batch can be computed in any order, as they are parallelizable computation units divided along parallelizable dimensions of the attention operator. However, block division determines communication scheduling, as each computation block may be associated with different amounts of communication from distinct devices. Ideally, we want divisions on every device to have balanced computation and communication load, as well as balancing among devices. Finding such divisions is equivalent to solving a multi-dimensional assignment problem (i.e., assigning computation blocks into T divisions while minimizing the maximum computation/communication on each device), which is NP-complete [15]. We devise a greedy heuristic to find balanced divisions (Listing 3).

For each device, we calculate the total required communication volume of its assigned computation blocks (total size of data sent to/received from other devices), and divide the workload such that each division accounts for $\frac{1}{T}$ of this total communication (Lines 12-14 in Listing 3). We schedule all computation blocks that do not require communication (Line 16-20) into the first division on each device. Then the device with the least computation load is selected and we schedule its second division by going through all the rest computation blocks on this device: if scheduling a block into this division causes the communication to exceed the per-stage requirement (i.e., $\frac{1}{T}$ of total communication), we defer the block to the following division; otherwise, we schedule the block into this division. We repeat the above procedure to schedule the second division on all devices. Similarly, the third, fourth, ..., $T - 1$ th divisions are scheduled consecutively on each device (Lines 28-35). We schedule all remaining blocks into the final D th division, regardless of their communication volumes (Lines 21-26). If the output block of a computation block on a device is placed on another device, we perform output transfer after all divisions and corresponding output reduction are performed on this device (Lines 36-38).

The communication and computation of each division, as well as the associated reduction operations, are serialized into an execution plan that contains a list of DCP instructions (each representing a fused operation) and is ready to be consumed by the executor.

5 EXECUTOR

DCP’s executor on each device adopts a block-centric design, consisting of two key components: block buffers and the instructions that operate on them.

Block buffers reside in GPU memory and hold all data blocks used by this device, including local input (Q , KV) and local output (O), data fetched from other devices, and intermediate results pending for reduction. To reduce memory fragmentation, a single contiguous buffer is used for all data blocks of the same type (e.g., all Q blocks). Each data block is thus identified by its type and its index into the buffer. The buffer index of each data block is determined during computation/communication scheduling (§4.3) by a buffer manager that keeps track of which buffer indices are occupied. We maximally reuse buffer indices that contain no longer needed blocks to minimize the total buffer size.

We abstract 5 types of DCP instructions for elementary operations in distributed attention execution:

- **Blockwise Attention** executes fused and masked attention (Listing 1 Line 5) specified by all computation blocks in a division. Especially, it takes a list of query, key-value,

Listing 3: Block Scheduling Pseudo-code

```

1 # Inputs:
2 # hg: partitioned hypergraph containing computation and data
3 #   blocks, with each block assigned to a device
4 # T: number of divisions
5 # devices: all devices participating in DCP
6 # Output: divs[division][device][comp or comm]
7 #   comp blocks and communication in each division
8 #   on each device
9 def schedule(hg, T, devices):
10     # comm_requirements[d1][d2]:
11     #   the total amount of data transfer from d2 to d1
12     comm_requirements = calc_required_comms(hg, devices)
13     for (d1, d2), comm_size in comm_requirements:
14         per_div_comm_limit[d1][d2] = comm_size / T
15     for i in {0, ..., T-1}:
16         if i == 0:
17             # first division
18             for d in devices:
19                 divs[i][d][comp] = comp_with_no_comm(hg, d)
20                 divs[i][d][comm] = None
21         elif i == T-1:
22             # last division
23             for d in devices:
24                 remaining_blocks = get_remaining_blocks(hg, divs, d)
25                 divs[i][d][comp] = remaining_blocks
26                 divs[i][d][comm] = get_comms(hg, remaining_blocks)
27         else:
28             while exists_schedulable_blocks:
29                 d = get_device_with_min_comp_load(hg, divs[i])
30                 for comp_block in get_remaining_blocks(hg, divs, d):
31                     if scheduling_comp_block_violates_per_div_comm_limit:
32                         continue
33                     else:
34                         divs[i][d][comp].append(comp_block)
35                         divs[i][d][comm].append(get_comms(comp_block))
36             # schedule communication for output blocks, if required
37         for d in devices:
38             divs[N][d][comm] = get_output_comms(hg, d)
39     return divs

```

and output tuples as input and performs attention on corresponding blocks. We base our attention implementation on FlashAttention [10]. Since the input and output blocks may not be contiguous in memory, we modify FlashAttention and enable all input/output blocks to be specified by the block buffer’s starting address and offsets of blocks recorded in block tables (similar to PagedAttention [26]). We support various attention masks via arrays specifying the index ranges each token should attend to, with the limitation of at most two ranges for each token (for simplicity of implementation). More flexible implementation is possible by adopting methods for optimizing sparse attention kernels like FlexAttention [12] and FlashMask [43], which is orthogonal to this paper.

- **Blockwise Reduction** takes multiple attention output blocks as input and performs fused update and reduction operation (Listing 1, Line 6) on these blocks. We implement this kernel with Triton [40].

- **Blockwise Copy** takes multiple input or output data blocks as input and performs fused GPU memory copy on a single device. This operation is used for managing buffers during DCP execution and is implemented with Triton [40].

- **Comm. Launch** takes a list of data blocks and asynchronously launches the communication to transfer these data blocks between devices. It is implemented with PyTorch’s P2P communication primitives [3], which internally use NCCL [29] as backend.

- **Comm. Wait** instructs the GPU to wait for a previously launched communication.

An execution plan consists of a sequence of such instructions with corresponding arguments. At runtime, the instructions in an execution plan are sequentially executed by the executor.

6 OTHER IMPLEMENTATION DETAILS

DCP’s core modules (dataloader, planner, executor) are implemented with 14k LOCs in Python, with an additional 300 LOCs for accelerating the computation/communication planning algorithm in C++.

6.1 Overlapping planning with model execution

The DCP dataloader pre-fetches input information for the planner to produce an execution plan for each training iteration asynchronously, well before the actual training iteration starts. Specifically, developers can define κ , the number of look-ahead iterations. The dataloader tries to ensure that when executing iteration i , the planning for iterations i through $i + \kappa$ is finished. Whenever this condition is not met, it pre-fetches input information for a new iteration and invokes a planner instance for planning. The planning of iterations i through $i + \kappa$ can be conducted in parallel. For multi-machine distributed training, we assign the execution planning (i.e., taking sequence lengths and attention masks as input and generating execution plans for each device) of different iterations to different machines. On each machine, the planning for different training iterations is parallelized to run on different CPU cores. The resulting execution plans are distributed to each device via a distributed key-value store (e.g., Redis [37]) which is located in host memory in one of the machines. This parallel execution planning using CPU resources greatly reduces the planning overhead, allowing us to overlap planning with actual model execution effectively.

6.2 Combining with other parallelisms

Data parallel training of different input sequences is included in parallelization configurations that can be produced by DCP. Tensor parallelism is orthogonal to DCP’s parallelization configurations, but it too partitions the head dimension of all attention input and output tensors (Q, KV, O). When jointly used with tensor parallelism, DCP’s head dimension size should be divided by the tensor parallel degree (i.e., the

number of devices that partition a tensor), and the same execution plan is shared among different tensor parallel groups. Pipeline parallelism is also orthogonal to DCP. It splits different model layers across stages, but each stage can still use context parallelism, where DCP’s optimizations can be applied. When used in conjunction with TP and PP, DCP should occupy the device ranks traditionally assigned to DP and CP (e.g., following the TP-CP-DP-PP rank assignment order in Megatron-LM [39]). For example, TP can be applied among consecutive ranks within a node to mitigate its high communication cost, followed by DCP which incurs less communication overhead than TP but more than PP, and finally, PP can be applied among distant ranks.

7 EVALUATION

We conduct extensive experimental evaluation of DCP in two steps: micro-benchmarking of the attention operator, and end-to-end evaluation of whole model training performance. We also evaluate the detailed performance of DCP across different parameters.

7.1 Attention micro-benchmarking

We first evaluate the distributed attention performance of DCP in detail.

Testbed. We conduct the micro-benchmarking experiments on 4 Amazon EC2 p4de.24xlarge instances. Each p4de instance is equipped with 8 NVIDIA A100 (80GB) GPUs and 96 vCPU cores. The 8 GPUs in each instance are connected by NVSwitch with 600GB/s bidirectional bandwidth. Inter-instance communication is supported by 4x100 Gbps NICs on each instance with EFA [2] enabled.

Baselines. We compare DCP with state-of-the-art long-context model training frameworks: (i) RingFlashAttention (RFA) [49], which parallelizes attention computation only at the sequence length dimension and supports two different input placements: Ring, which splits each input sequence into R blocks (R is the number of devices) and places the r th block of the sequences onto the r th device; ZigZag, which splits each sequence into $2R$ blocks and uses a zig-zag pattern for block-to-device assignment (such a placement pattern is also used by the following two baselines (§2.2)). (ii) LoongTrain (LT) [20] parallelizes attention at both head and sequence length dimensions and does not support variable-length input sequences, so we pad the sequences to the longest sequence length in each batch. It requires specifying the size of the inner ring for its double ring attention (a communication schedule that aims to improve NIC utilization), and we experiment under inner-ring sizes 1,2,4,8 and report the best result. (iii) TransformerEngine (TE) [30] also parallelizes attention at both head and sequence length dimensions and lacks variable-length input support when parallelizing at

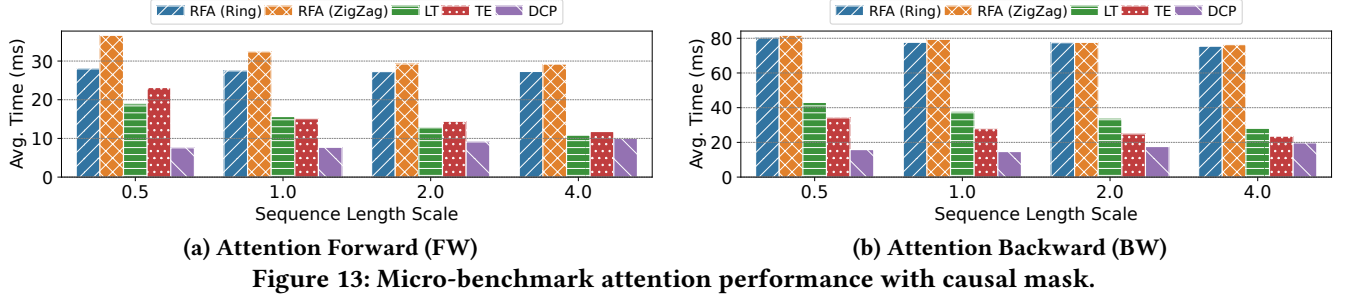


Figure 13: Micro-benchmark attention performance with causal mask.

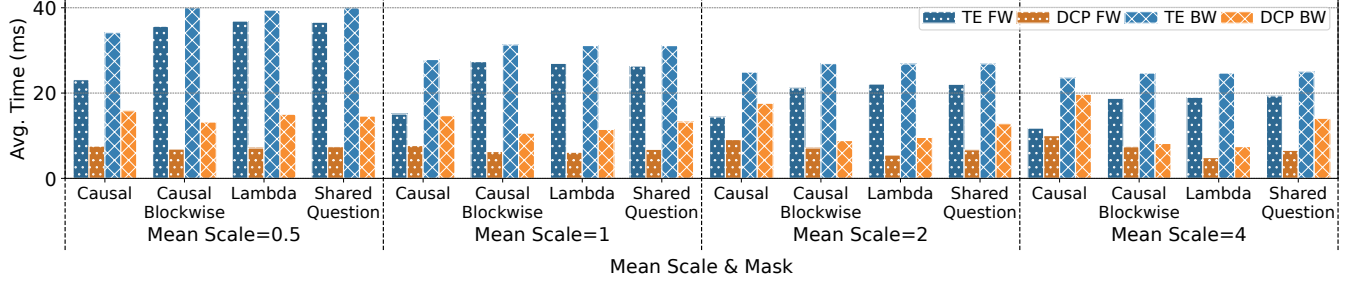


Figure 14: Micro-benchmark attention performance under different attention masks.

both dimensions, but can be easily extended to accommodate it. None of the baselines support attention masks other than causal masks. To facilitate a direct comparison, we add this support to TransformerEngine by (pre-)computing a local mask for each of its computation steps and use DCP’s masked attention kernels, without changing its communication pattern.

Hyper-parameters. DCP requires a few hyper-parameters: block size $\mathcal{B}$ (that we use to partition the sequence length dimension), the number of divisions for each input batch in computation/communication planning (§4.3), and the computation imbalance tolerance (ϵ). We empirically fix the number of divisions to 4 since it yields overall good performance in our test scenarios. We search through block sizes 512, 1024, 2048, 4096 and report the best performance, and use inter-node $\epsilon = 0.4$ and intra-node $\epsilon = 0.1$ in all our experiments (except in the ablation study in Sec. 7.3). We use KaHyPar [38] as our hypergraph partitioning solver.

Dataset. We use the LongDataCollections [41] dataset, a compilation of common long-context datasets designed for long input understanding tasks. It exhibits skewed, long-tailed sequence length distributions similar to those found in larger pre-training datasets like the Pile [16] (Fig. 2). Such a sequence length distribution pattern is also observed in other studies [18, 45]. Additionally, to better understand the influence of the number of short sequences on the performance, we obtain four variations by multiplying the length of each sequence by 0.5, 1 (no scaling), 2 and 4. We use a global batch size of 131072 tokens, and set the maximally

allowed sequence length to the same value. We report the average attention execution time over the first 200 batches.

Attention Masks. We implement four types of attention masks: causal mask, causal blockwise mask [6], lambda (Λ) mask [21, 27], and the shared question mask [33, 43]. Lambda mask uses 64 attention sink tokens and a window size of 4096. Causal blockwise masking uses a fixed block size of 256, a sliding window of 2 blocks, and a single block for both the attention sink and test sample. Shared question mask assumes a single shared question with 4 different answers, each taking up 20% of the entire sequence length. The same mask specification is used in the end-to-end evaluation as well.

Attention Op. Spec. We use GQA [1], with 8 total heads for Q, 2 groups for KV and head dimension 128 (corresponding to a 32 head, 8 KV group attention operation using 4-way tensor parallelism on the head dimension). All 32 GPUs are used in context parallelism. For baselines that support parallelizing both head and sequence length dimensions, we set the head parallelization size to 2 (the number of KV groups) to minimize their communication.

Fig. 13 shows the average execution time of the forward pass and the backward pass of the attention operator when using the causal attention mask, under different sequence length scalings. DCP achieves the highest speed-up in most cases. DCP dynamically computes the parallelization configurations according to input sequence lengths and can place an entire short sequence on one device to minimize communication, while the baselines require nearly the same communication for all batches; thus DCP’s speed-up is highest when

there are more short sequences (at sequence length scale 0.5), 2.45x (considering both forward and backward) compared to the next best baseline (LT). As the sequence length scale increases, DCP's attention time becomes close to LT or TE's, since there are little opportunities for DCP's parallelization optimization as compared to baselines, for batches consisting of only a small number of long sequences. RFA performs the worst overall, as it does not support parallelizing along the head dimension, resulting in significantly higher communication costs. LT's performance improves with increasing sequence length scale due to the larger amount of padding in batches of variable-length sequences. Similarly, TE's performance improves as sequence length scale increases, although it does not rely on padding. We suspect this is due to the setting being communication bound (attention computation time is small), while overheads (e.g., reordering tensors between head and ring parallelization, constructing attention arguments) decrease with the number of sequences.

Fig. 14 shows the performance of DCP and TE under different attention masks. On sparse masks, DCP significantly outperforms TE at up to 3.77x, since DCP removes redundant communication with its fine-grained block partitioning and placement. The speed-up is more significant on causal blockwise and lambda masks compared with shared question mask, since the former two exhibit more sparsity.

7.2 End-to-end evaluation

We first evaluate end-to-end model training performance with DCP.

Testbed. We conduct end-to-end experiments on eight Amazon EC2 p4de.24xlarge instances (64 GPUs in total). While the instances are the same as those used in the micro-benchmarks, a larger scale is employed to accommodate the high memory demands of end-to-end model training.

Baseline. Our models are implemented in Megatron-LM (MLM) [39], a highly regarded framework for training models with state-of-the-art performance. We replace the attention module in Megatron-LM with DCP's executor. For the baseline, Megatron-LM natively supports context parallelism through TransformerEngine [30]; we use our enhanced TE (supporting variable-length inputs and attention masks) in its place as the baseline.

Dataset. In addition to the LongDataCollections [41] dataset used in §7.1, we experiment on the LongAlign [5] dataset, which is used for long-context LLM alignment (post-training). It has longer average sequence lengths and fewer short sequences compared to LongDataCollections, while exhibiting similar sequence length distribution patterns (Fig. 2).

Model Spec. We implement a GPT (8B) [35] model with 32 layers, hidden size 4096, 32 heads, 8 KV groups, head dimension 128 and FFN hidden size 14336 (corresponding to the setup of Llama3-8B [13]). We use 4-way tensor parallelism within each instance, and 16-way context parallelism among the rest of the devices.

Fig. 15 and Fig. 16 show the per-iteration training time on LongAlign and LongDataCollections datasets, respectively. Overall, DCP achieves up to 1.16x speed-up when using the causal mask, and 1.46x under sparse masks. The speed-up appears smaller compared to those in micro-benchmark experiments since the execution time of context-independent operators and the time needed for gradient synchronization is similar for both DCP and MLM baseline. For the causal mask, the speed-up is higher on the LongDataCollections dataset, since it has more short sequences. On the LongAlign dataset, DCP can underperform MLM under the causal mask when the maximum sequence length is large. We attribute this to less overlap between computation and communication (analyzed in § 7.5). The speed-up under causal mask is also higher at smaller maximum sequence lengths, since shorter maximum sequence lengths exclude long sequences and lead to higher percentages of short sequences in batches. We also observe consistent speed-ups with sparse attention masks, with higher speed-ups under lambda and causal blockwise masks, aligning with observations in §7.1.

7.3 Further Performance Analysis

We further analyze the performance of DCP in detail. We use the same experimental setups as in the end-to-end experiment (§7.2).

Communication vs. block size. Fig. 17 shows the impact of the block size on total inter-instance communication volume and maximum per-device communication volume (including send and receive). DCP requires much less communication than the MLM baseline (Fig. 17a,17b), with communication volume slightly increasing with the block size, since a larger block size (and thus a smaller total block number) provides less placement flexibility.

Planning time vs. block size. Fig. 18 shows the influence of the block size on planning time (including block generation, hypergraph partitioning and computation/communication scheduling). As the block size increases, planning time rapidly decreases since the time is directly related to the total number of blocks. For similar reasons, the planning time is much smaller under sparse attention masks. When choosing a reasonable block size, the average planning time is less than 10 seconds per training batch/iteration, which can perfectly overlap model execution time (> 1 second per iteration) using our pre-fetching and parallel planning design if planning is parallelized with more than 10 CPU cores

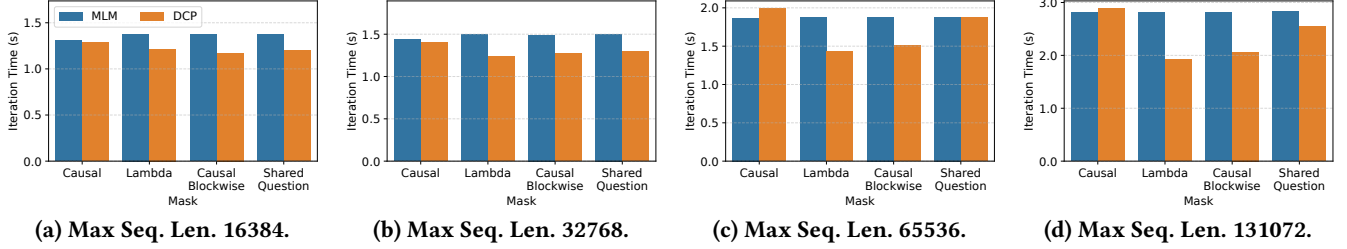


Figure 15: End-to-end training performance on the LongAlign dataset.

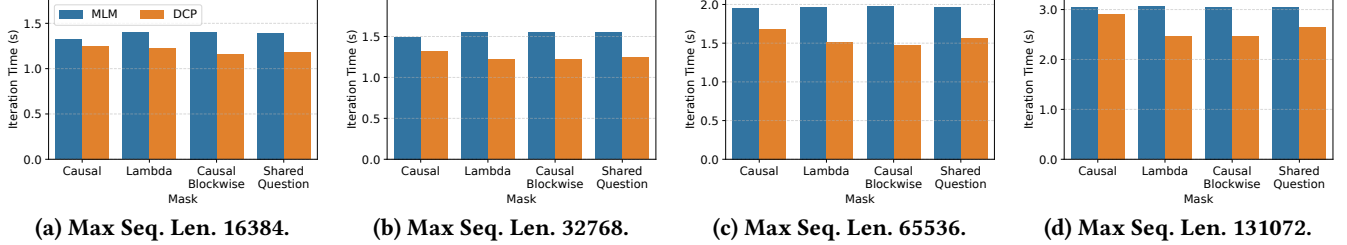


Figure 16: End-to-end training performance on the LongDataCollections dataset.

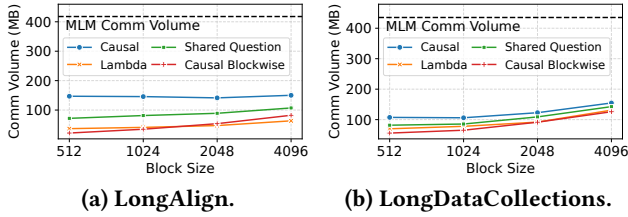


Figure 17: Total inter-node communication volume v.s. block size.

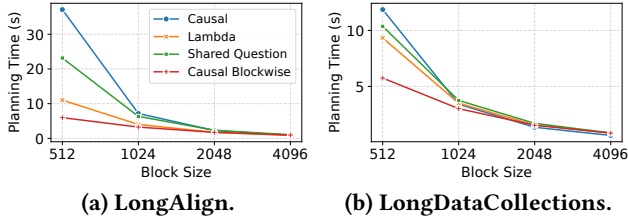


Figure 18: Impact of block size on planning time.

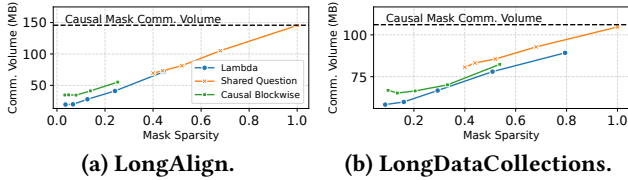


Figure 19: Impact of mask sparsity on communication.

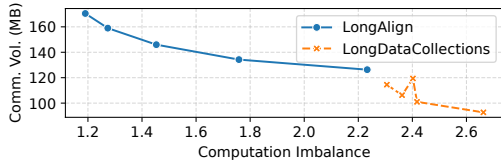


Figure 20: Impact of computation imbalance tolerance on communication.

(much less than the available number of cores for a typical training server, e.g., 96 as in AWS EC2 p4de.24xlarge).

Communication vs. mask sparsity. In Fig. 19, the mask sparsity is computed as the amount of computation (FLOPS) required for the sparse mask divided by that of the causal mask. We observe that the required communication volume with DCP grows nearly linearly with mask sparsity, indicating that DCP can exploit the mask sparsity well in eliminating redundant communication.

Communication vs. computation imbalance tolerance. In Fig. 20, the required communication decreases as we increase computation imbalance tolerance (ϵ), indicating a clear trade-off between computation imbalance and required communication. In communication-bound scenarios, ϵ should be set to a larger value, and vice versa.

7.4 Precision

Since DCP does not alter the attention algorithm, we expect it to have no impact on training accuracy. To verify this, we compare the training loss curves of DCP with those of standard head and ring parallel attention, as implemented in the MLM baseline. We use the same setup as in the end-to-end experiments. In Fig. 21, DCP's loss curve matches that of the MLM baseline, only with small deviations due to different kernel implementations and attention/reduction computation orders.

7.5 Speed-up decomposition

Fig. 22 illustrates the decomposition of end-to-end iteration time, computed from NVIDIA Nsight Systems[31] traces

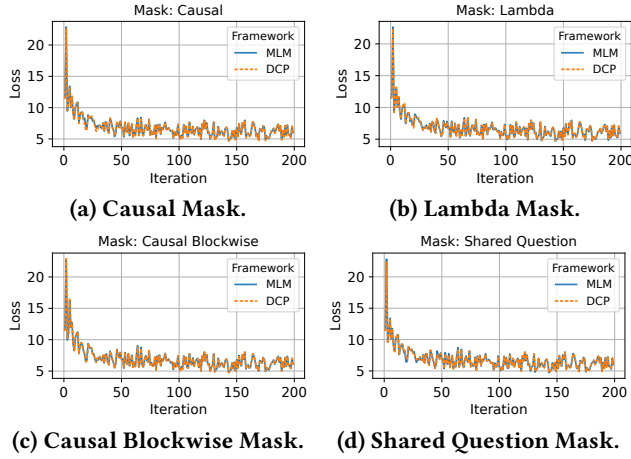


Figure 21: Training loss curves on the LongAlign dataset (max. sequence length 131072).

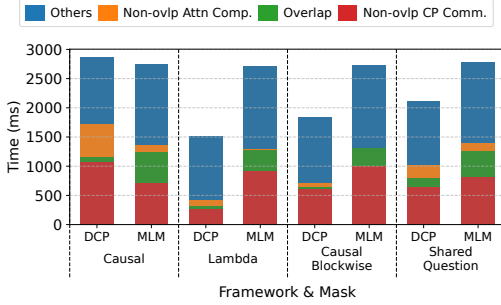


Figure 22: Decomposition of end-to-end iteration time (LongAlign dataset, max sequence length 131072).

collected during iterations 50–55, with the same setup as Fig.15d. For sparse masks, DCP significantly reduces the total communication time (i.e., Non-overlap CP communication + Overlap). The total attention computation time is also slightly reduced, especially for highly sparse lambda and causal-blockwise masks. Most of this reduction comes from the attention backward kernel: MLM uses a fixed number of backward steps, each incurring a certain overhead (i.e., reading local Q and KV from memory, writing gradients back, and performing additional gradient reduction operations across different blocks). This overhead is more significant in the backward pass. We observe that DCP’s scheduler tends to concentrate all backward computation into one or two divisions (see Section 4.3) when using sparse masks, thus reducing the overall overhead and computation time. In the causal mask case, where DCP slightly underperforms MLM, the total communication time is still reduced; however, the overlap between computation and communication decreases noticeably. We attribute this to limitations in the scheduling algorithm and believe further research could improve its performance.

8 DISCUSSION AND RELATED WORKS

Scaling to larger models/clusters. As model or cluster size increases, DCP’s planning overhead is expected to scale sub-linearly for a given input: 1) planning is independent of model size; 2) graph partitioning primarily depends on the number of input blocks, not cluster size; and 3) greedy scheduling scales linearly with the number of devices. Thus the planning-to-execution time ratio decreases with system scaling. Batch size scaling can be managed by adding CPU resources or grouping nodes, applying DCP within groups and traditional DP across groups. Performance-wise, DCP’s optimizations are unaffected by hidden size or layer count, as layers share the same structure. Expanding context parallelism to more devices increases communication overhead, underscoring the importance of our communication optimizations.

Related Works. Current context-parallel frameworks like RingAttention [28], USP [14], LoongTrain [20] and TransformerEngine [30] employ a static parallelization configuration that does not adapt to input dynamism. Hierarchical Balance Packing[48] and WLB-LLM [45] mitigate imbalanced computation within data and pipeline parallelism caused by input sequence length variance via optimizing packing algorithms. WLB-LLM further discusses the imbalanced computation in context parallelism caused by applying partitioning directly on packed inputs. In this work, our discussion assumes context parallelism partitioning is performed at the sequence (i.e., each sample/document in the batch) level, which does not exhibit such imbalance. ByteScale [18] and FlexSP [44] allow different sequences to be parallelized differently (DP v.s. CP/SP) to minimize communication, which is similar to our objective. However, they do not model fine-grained token dependencies and thus do not support various sparse or structured attention patterns.

9 CONCLUSION

We propose DCP, a context parallel training framework optimized for input dynamism. We enable fine-grained parallelism control for each input sequence, optimize data and computation placement via hypergraph partitioning and design efficient planner and executor modules to minimize planning and execution overhead. Extensive evaluation shows that DCP achieves up to 0.94x~1.16x end-to-end speed-up training a model with causal mask, and 1.00x~1.46x with sparse masks.

ACKNOWLEDGMENTS

We would like to thank the anonymous reviewers and our shepherd Tim Harris for their valuable feedback. This work was supported in part by grants from Hong Kong RGC under the contracts 17204423, 17205824, and C7004-22G (CRF).

REFERENCES

- [1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *EMNLP'23*, 2023.
- [2] Amazon Web Services, Inc. Elastic Fabric Adapter. <https://aws.amazon.com/hpc/efa/>, 2023.
- [3] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhresch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *ASPLOS'24*, 2024.
- [4] Anthropic. The claude 3 model family: Opus, sonnet, haiku. https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf, 2025.
- [5] Yushi Bai, Xin Lv, Jiajie Zhang, Yuze He, Ji Qi, Lei Hou, Jie Tang, Yuxiao Dong, and Juanzi Li. LongAlign: A recipe for long context alignment of large language models. In *EMNLP'24*, 2024.
- [6] Amanda Bertsch, Maor Ivgi, Emily Xiao, Uri Alon, Jonathan Berant, Matthew R. Gormley, and Graham Neubig. In-context learning with long-context models: An in-depth exploration. *arXiv 2405.00200 [cs.CL]*, 2025.
- [7] Ümit Çatalyürek and Cevdet Aykanat. PaToH (partitioning tool for hypergraphs). In *Encyclopedia of Parallel Computing*, pages 1479–1487, 2011.
- [8] Ümit V. Çatalyürek and Cevdet Aykanat. Decomposing irregularly sparse matrices for parallel matrix-vector multiplication. In *Parallel Algorithms for Irregularly Structured Problems*, pages 75–86, 1996.
- [9] U.V. Catalyurek and C. Aykanat. Hypergraph-partitioning-based decomposition for parallel sparse-matrix vector multiplication. *IEEE Transactions on Parallel and Distributed Systems*, 1999.
- [10] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *arXiv 2205.14135 [cs.LG]*, 2022.
- [11] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv 2501.12948 [cs.CL]*, 2025.
- [12] Juechu Dong, Boyuan Feng, Driss Guessous, Yanbo Liang, and Horace He. Flex attention: A programming model for generating optimized attention kernels. *arXiv 2412.05496 [cs.LG]*, 2024.
- [13] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv 2407.21783 [cs.AI]*, 2024.
- [14] Jiarui Fang and Shangchun Zhao. USP: A unified sequence parallelism approach for long context generative ai. *arXiv 2405.07719 [cs.LG]*, 2024.
- [15] A.M. Frieze. Complexity of a 3-dimensional assignment problem. *European Journal of Operational Research*, 13(2):161–164, 1983.
- [16] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling. *arXiv 2101.00027 [cs.CL]*, 2020.
- [17] M.R. Garey, D.S. Johnson, and L. Stockmeyer. Some simplified np-complete graph problems. *Theoretical Computer Science*, 1(3):237–267, 1976.
- [18] Hao Ge, Junda Feng, Qi Huang, Fangcheng Fu, Xiaonan Nie, Lei Zuo, Haibin Lin, Bin Cui, and Xin Liu. Bytescale: Efficient scaling of llm training with a 2048k context length on more than 12,000 gpus. *arXiv 2502.21231 [cs.DC]*, 2025.
- [19] Google Gemini Team. Gemini 2.5: Our most intelligent ai model. <https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/>, 2025.
- [20] Diandian Gu, Peng Sun, Qinghao Hu, Ting Huang, Xun Chen, Yingdong Xiong, Guoteng Wang, Qiaoling Chen, Shangchun Zhao, Jiarui Fang, Yonggang Wen, Tianwei Zhang, Xin Jin, and Xuanzhe Liu. LoongTrain: Efficient training of long-sequence llms with head-context parallelism. *arXiv 2406.18485 [cs.DC]*, 2024.
- [21] Chi Han, Qifan Wang, Hao Peng, Wenhan Xiong, Yu Chen, Heng Ji, and Sinong Wang. LM-infinite: Zero-shot extreme length generalization for large language models. In *NAACL'24*, 2024.
- [22] Amazon Artificial General Intelligence. The amazon nova family of models: Technical report and model card. 2024.
- [23] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. DeepSpeed Ulysses: System optimizations for enabling training of extreme long sequence transformer models. *arXiv 2309.14509 [cs.LG]*, 2023.
- [24] Chenyu Jiang, Zhen Jia, Shuai Zheng, Yida Wang, and Chuan Wu. Dynapipeline: Optimizing multi-task training through dynamic pipelines. In *EuroSys '24*, 2024.
- [25] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing activation recomputation in large transformer models. In *MLSys'23*, 2023.
- [26] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *SOSP'23*, page 611–626, 2023.
- [27] Bin Lin, Chen Zhang, Tao Peng, Hanyu Zhao, Wencong Xiao, Minmin Sun, Anmin Liu, Zhipeng Zhang, Lanbo Li, Xiafei Qiu, Shen Li, Zhigang Ji, Tao Xie, Yong Li, and Wei Lin. Infinite-LLM: Efficient llm service for long context with DistAttention and distributed KVCache. *arXiv 2401.02669 [cs.DC]*, 2024.
- [28] Hao Liu, Matei Zaharia, and Pieter Abbeel. RingAttention with block-wise transformers for near-infinite context. In *ICLR'24*, 2024.
- [29] NVIDIA. NCCL: Optimized primitives for inter-gpu communication. <https://github.com/NVIDIA/nccl>, 2024.
- [30] NVIDIA. Transformer Engine. <https://github.com/NVIDIA/TransformerEngine>, 2024.
- [31] NVIDIA. NVIDIA Nsight Systems. <https://developer.nvidia.com/nsight-systems>, 2025.
- [32] OpenAI. Gpt-4o system card. *arXiv 2410.21276 [cs.CL]*, 2024.
- [33] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *NeurIPS'22*, 2022.
- [34] Markus N. Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv 2112.05682 [cs.LG]*, 2022.
- [35] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.
- [36] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *NeurIPS'23*, 2023.
- [37] Redis. Redis - the real-time data platform. <https://redis.io/>.

- [38] Sebastian Schlag. *High-Quality Hypergraph Partitioning*. PhD thesis, Karlsruhe Institute of Technology, Germany, 2020.
- [39] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv 1909.08053 [cs.CL]*, 2020.
- [40] Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *MAPL'19*, page 10–19, 2019.
- [41] together.ai. togethercomputer/Long-Data-Collections. <https://huggingface.co/datasets/togethercomputer/Long-Data-Collections>, 2024.
- [42] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS'17*, 2017.
- [43] Guoxia Wang, Jinle Zeng, Xiyuan Xiao, Siming Wu, Jiabin Yang, Lujing Zheng, Zeyu Chen, Jiang Bian, Dianhai Yu, and Haifeng Wang. Flashmask: Efficient and rich mask extension of flashattention. In *ICLR'25*, 2025.
- [44] Yujie Wang, Shiju Wang, Shenhan Zhu, Fangcheng Fu, Xinyi Liu, Xuefeng Xiao, Huixia Li, Jiashi Li, Faming Wu, and Bin Cui. Flexsp: Accelerating large language model training via flexible sequence parallelism. In *ASPLOS'25*, 2025.
- [45] Zheng Wang, Anna Cai, Xinfeng Xie, Zaifeng Pan, Yue Guan, Weiwei Chu, Jie Wang, Shikai Li, Jianyu Huang, Chris Cai, Yuchen Hao, and Yufei Ding. Wlb-llm: Workload-balanced 4d parallelism for large language model training. *arXiv 2503.17924 [cs.DC]*, 2025.
- [46] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *ICLR'24*, 2024.
- [47] Fuzhao Xue, Yukang Chen, Dacheng Li, Qinghao Hu, Ligeng Zhu, Xiuyu Li, Yunhao Fang, Haotian Tang, Shang Yang, Zhijian Liu, Ethan He, Hongxu Yin, Pavlo Molchanov, Jan Kautz, Linxi Fan, Yuke Zhu, Yao Lu, and Song Han. Longvila: Scaling long-context visual language models for long videos. *arXiv 2408.10188 [cs.CV]*, 2024.
- [48] Yongqiang Yao, Jingru Tan, Kaihuan Liang, Feizhao Zhang, Yazhe Niu, Jiahao Hu, Ruihao Gong, Dahua Lin, and Ningyi Xu. Hierarchical balance packing: Towards efficient supervised fine-tuning for long-context llm. *arXiv 2503.07680 [cs.LG]*, 2025.
- [49] Zilin Zhu. Ring Flash Attention. <https://github.com/zhuzilin/ring-flash-attention>, 2024.